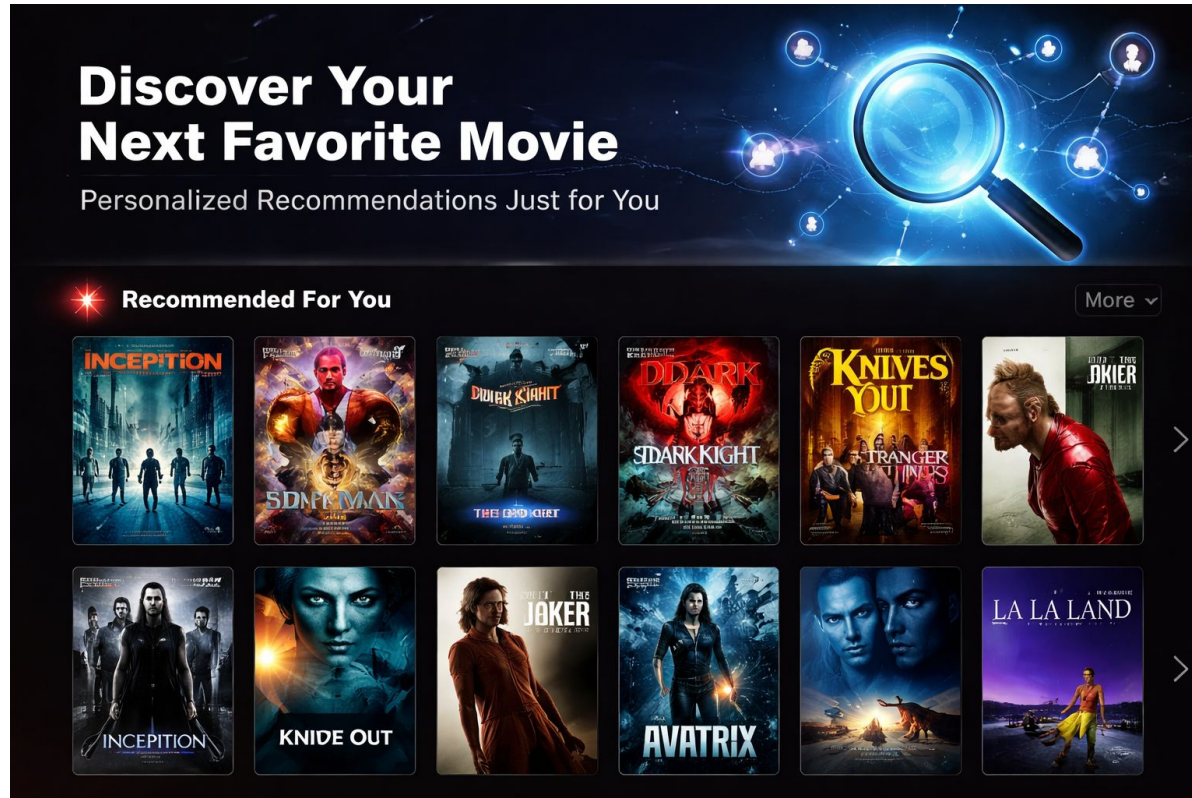


Content-Based Movie Recommendations Using Text Embeddings Into High Dimensional Vectors

Rich Shelby
Andrea Hwang
Devashish Singh

Business Scenario: Personalized Movie Recommendations

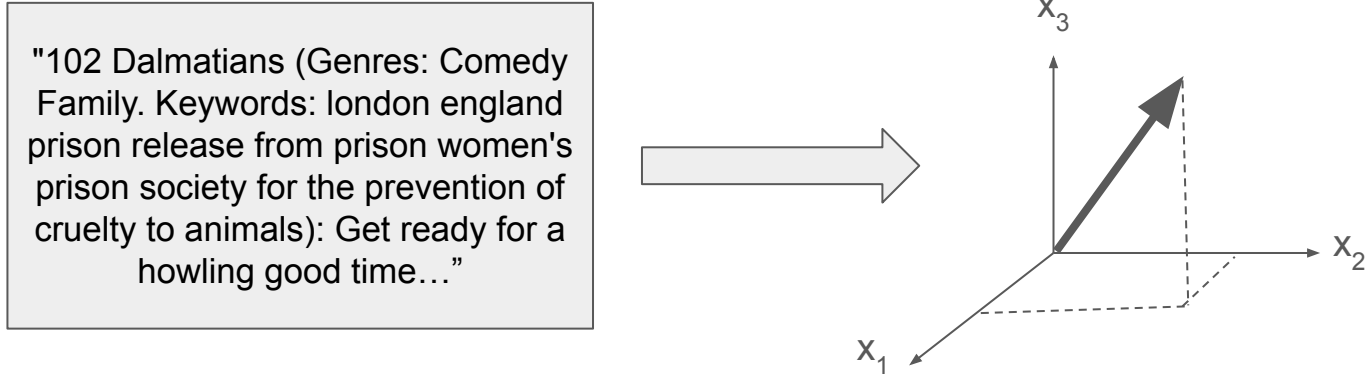


Graph (Cleaning)

- Original data frame shape: 4803 rows x 24 columns
- Rows dropped if:
 - Numeric columns (i.e. budget, revenue, average vote, total vote, runtime) were empty or 0
 - Director or cast was missing
- Crew column was a nested JSON which was extracted for each row and converted into executive producer, music composer, director of photography and writer columns
 - Similar process done for production company, languages and production country
- Additional columns not relevant to the development of the vector embeddings were dropped (i.e. homepage)
- Final data frame shape: 3223 rows x 18 columns

Graph (Vector embeddings)

- Sentence transformers to embed summarized text to high-dimensional (384-feature) vectors^{1, 2}



- Calculate distances between movies by taking cosine similarity scores

¹ <https://sbert.net/>

² O. I. I. R, C. E. P. B, J. O. López and C. G. Q. M, "Designing a Movie Recommendation System Through a Transformer-Based Embeddings Space," 2024 IEEE Colombian Conference on Communications and Computing (COLCOM), Barranquilla, Colombia, 2024, pp. 1-6, <https://ieeexplore.ieee.org/document/10720246>.

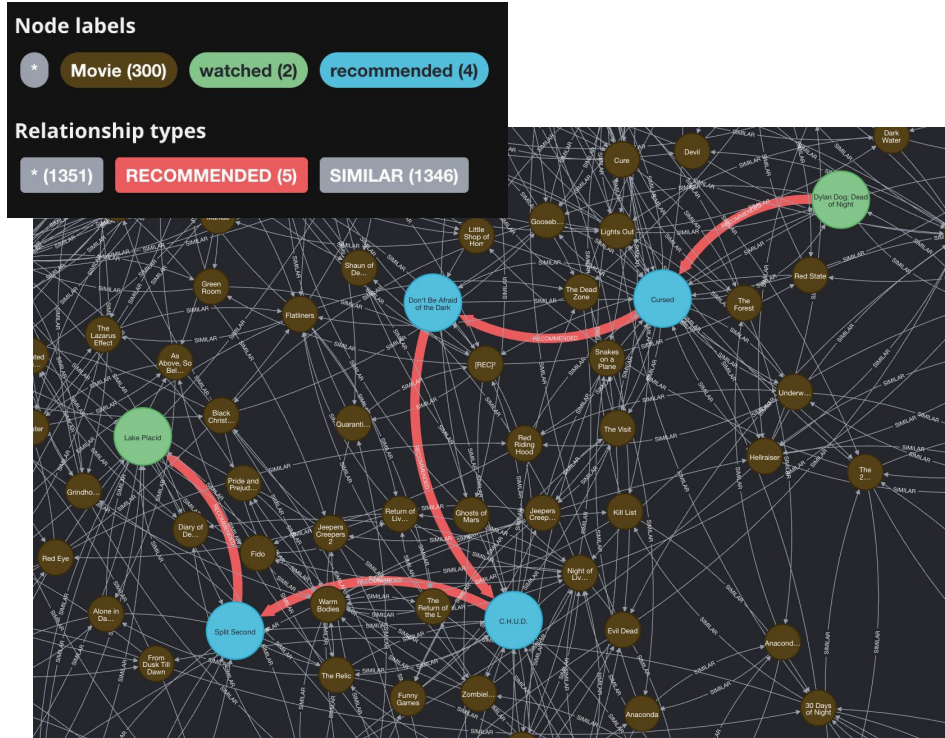
Graph of Movie Database

- Nodes: Movie attributes
- Edges: Similarity scores
 - Determine top 3 for each node



Subset of the graph

Graph Algorithm: Shortest Path (Dijkstra)

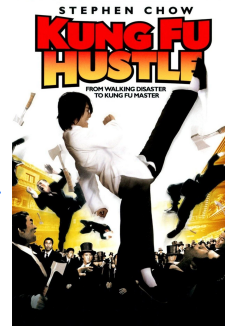


- Convert similarity into “cost”
$$\text{cost} = 1 - \text{similarity}$$
- Low cost = High similarity
- High cost = Low similarity
- Use Dijkstra to find lowest cost path between watched films

Personalized Pagerank

- Pagerank on similarity scores
 - Centered on a watched movie
- Output ordered by pagerank score, movie popularity (ascending) and movie rating (descending)
- Result is recommendations of influential movies that were highly rated but not highly viewed

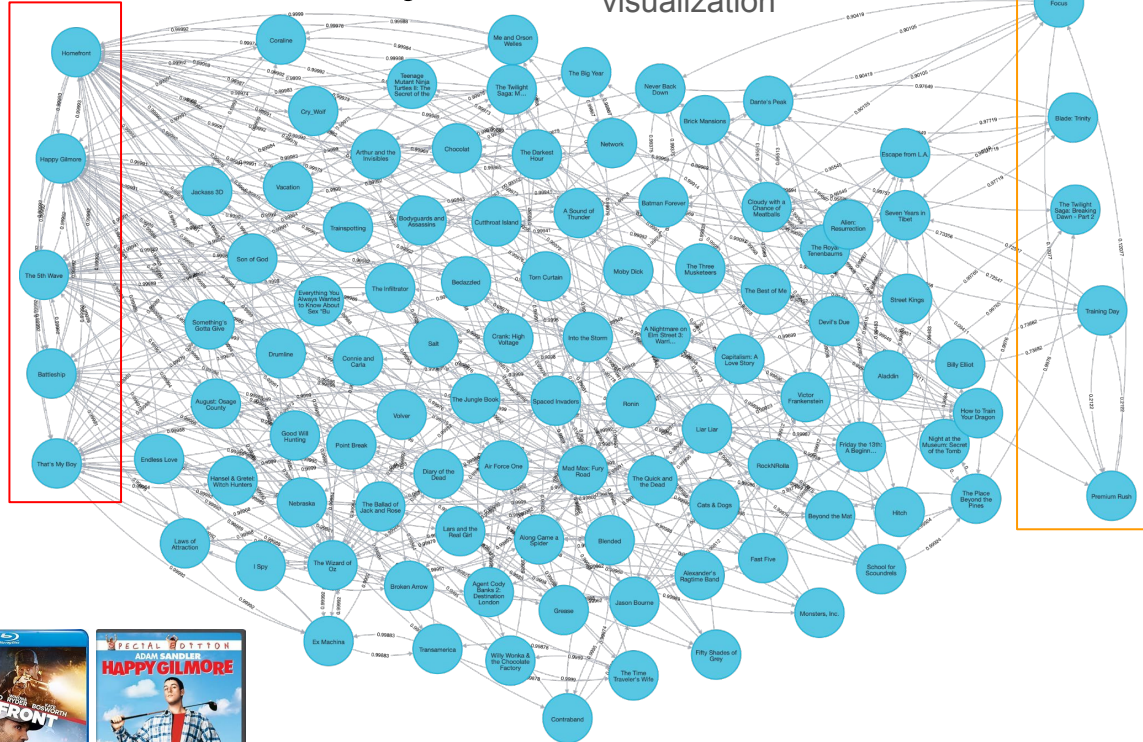
	title	movie_popularity	movie_rating	page_rank_score
0	Kung Fu Panda 2	51.247321	6.7	0.532033
1	Kung Fu Hustle	13.658985	7.2	0.112337
2	Kung Fu Panda	84.689648	6.9	0.112337
3	The Monkey King 2	4.726290	6.0	0.094780
4	Kundun	7.039189	6.8	0.033886
...	...	...	...	...
3218	The Hunger Games: Mockingjay - Part 1	206.227151	6.6	0.000000
3219	Pirates of the Caribbean: The Curse of the Bla...	271.972889	7.5	0.000000
3220	Mad Max: Fury Road	434.278564	7.2	0.000000
3221	Guardians of the Galaxy	481.098624	7.9	0.000000
3222	Deadpool	514.569956	7.4	0.000000



Graph Algorithm: Harmonic Centrality

- Measures closeness of a movie relative to others
- Streamline recommendations

Sampled 100 movies for visualization



	movie_title	closeness
0	Homefront	0.595960
1	Happy Gilmore	0.583670
2	The 5th Wave	0.515152
3	Battleship	0.512626
4	That's My Boy	0.507576

...	...	...
95	Focus	0.321212
96	Blade: Trinity	0.321212
97	The Twilight Saga: Breaking Dawn - Part 2	0.310774
98	Training Day	0.303199
99	Premium Rush	0.273401



Very different movies likely to be recommended

A case for MongoDB

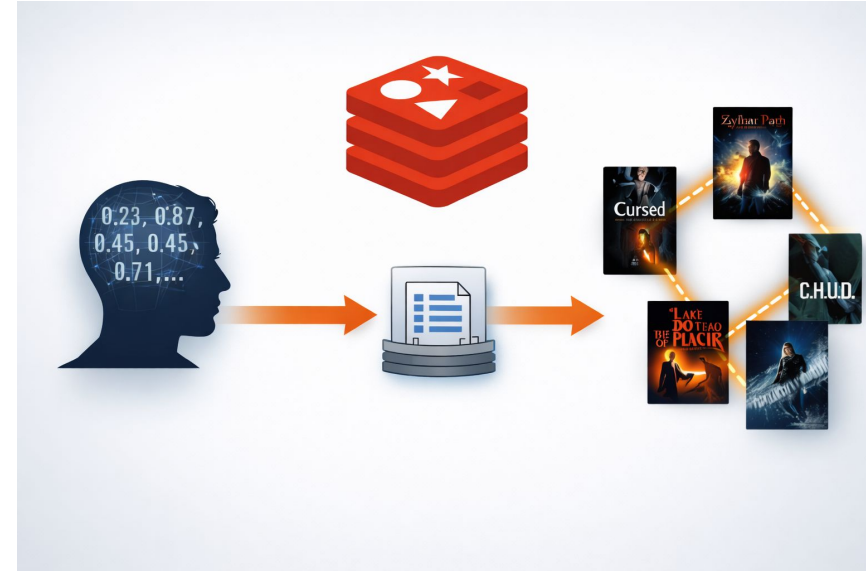
- Business case: Website with many users looking for individual movie recommendations
- Use MongoDB to
 - Create a document database for movie data used to create graphs, vector embeddings
 - Create a user POV to allow for personalized modulation of graph
 - Incorporate watch history, user ratings, user comments etc.
 - Pull user document on log in to provide accurate recommendations
 - Repeat graph algorithms incorporating customer data
- Improves on a relational database by allowing for quicker access of user data without the expense of building a multi-user relational database

Upscale with Redis

- Store precomputed embeddings
- Include user profile (watched / liked)
- Ultra fast similarity searching

Improvements over Relational Database

- Pairwise similarity expensive at scale
- Less pressure on primary database



What's Next?

Challenges

- Similarity scores were heavily influenced by formatting of text
- No key property weights in vector embeddings
- Poor genre granularity in the dataset
 - “Horror” and “Science Fiction” -> “Horror Science Fiction”

Future Work

- Deeper data collection
- Multiple graphs based on genre / user preferences
- Calculating weighted similarity scores for each key property

Thank you

Open to questions / feedback.

References

- Image Slide 2 (Business Scenario): OpenAI. (2026). AI-generated image using DALL·E (via ChatGPT).
- Images Slide 7:
 - https://encrypted-tbn3.gstatic.com/images?q=tbn:ANd9GcSXdan0emNi9od879ZddNwo_x2JlxPRrQa_2DEcayO8dRRrXpyQ
 - https://www.rottentomatoes.com/m/kung_fu_hustle
- Images slide 8:
 - <https://www.amazon.com/Homefront-Blu-ray-Jason-Statham/dp/B08N1K1W8N>
 - <https://www.amazon.com/Happy-Gilmore-Full-Screen-Special/dp/B0009X7614>
- Image Slide 10 (Redis): OpenAI. (2026). AI-generated image using DALL·E (via ChatGPT).